

ForL0 State Backend设计说明书

v2.0

文档类型： 项目交付文档
文档属性： 设计说明书
适用对象： 项目评审、研发交接、交付验收
版本日期： 2026年4月

项目名称： ForL0 State Backend

目 录

1 整体架构设计	3
1.1 总体架构	3
1.2 状态访问优化路径	3
1.3 原生状态表组织	4
1.4 组件解耦与集成	4
2 架构设计细节	6
2.1 可复用组件与扩展组件	6
2.1.1 ForL0StateBackend 与 ForL0KeyedStateBackend	6
2.1.2 PriorityQueueManager / HeapPriorityQueue	6
2.1.3 NativeEngine	6
2.2 关键设计组件	7
2.2.1 StateEngine	7
2.2.2 状态布局与命名空间组织	7
2.2.3 SwissTable	7
2.2.4 状态访问策略与 JNI 快路径	8
2.2.5 类型分析与布局描述	9
2.3 原生状态组织实现	9
2.3.1 结构组成	9
2.3.2 RowData 快路径实现	9
2.3.3 主要操作算法	10
2.4 ForL0State 实现	13
2.4.1 ForL0ValueState	13
2.4.2 ForL0ListState	13
2.4.3 ForL0MapState	13
2.4.4 ForL0ReducingState 与 ForL0AggregatingState	14
3 容错机制设计	14
3.1 快照发起	14
3.2 快照采集	14
3.3 Copy-On-Write 一致性模型	15

3.4	Savepoint 与状态恢复	15
3.5	资源回收与异常处理	15
4	插件式集成设计	16
4.1	加载机制	16
4.2	运行时接入边界	16
5	总结	17

1. 整体架构设计

ForL0 State Backend 面向 Apache Flink 的 Keyed State 场景设计，目标是在保持 Flink 状态接口、命名空间语义和 Checkpoint 机制兼容的前提下，将状态访问的主要数据路径收敛到一套原生状态引擎中，通过类型特化、缓存友好的哈希组织和低复制快照机制，降低状态访问延迟并提升整体处理效率。

与早期版本相比，当前实现已经从“Java 侧分层 StateStore 主导”的结构演进为“Java 控制层 + JNI 桥接层 + C++ 原生状态引擎”的三层架构。Java 层承担接口适配、状态注册和用户函数执行，native 层承担状态组织、命名空间隔离、哈希存储以及快照恢复。本文档仅说明系统设计本身，不再展开构建、部署和配置细节；相关内容由《编译构建指导书》和《使用说明书》单独说明。

1.1 总体架构

系统围绕 `StateEngine -> StateTable -> KeyGroup -> Namespace -> SwissTable` 的层级展开。一个任务实例对应一个 native `StateEngine`；每个状态描述符在 native 侧注册为一个 `typed StateTable<K,V>`；每个 `StateTable` 再按 `key-group` 和 `namespace` 组织具体状态条目；最底层的键值映射由 `SwissTable<K,V>` 完成。

整体职责可概括如下：

- Java 控制层负责与 Flink 运行时对接，管理状态对象生命周期、serializer 分析、状态注册和快照调度。
- JNI 桥接层负责共享库装载、handle 透传以及按类型划分的 native 调用入口。
- C++ 原生数据层负责状态条目的权威存储、哈希组织、命名空间管理、Copy-On-Write 快照和恢复写入。

这一结构的关键意义在于，状态主数据面已经完全位于 native 侧，Java 层不再维护一套独立的状态表副本，也不再承担状态快照的主序列化职责。

1.2 状态访问优化路径

系统专门设计了一层状态访问优化路径，用于缩短热点场景上的分支判断、对象封装和序列化链路。该优化路径不是独立的表结构，而是由以下机制共同构成：

- Java 状态对象在初始化阶段预计算访问策略，将 `key`、`namespace` 与 `value` 的组合固化为固定分支。
- `NativeEngine` 按高频类型组合暴露窄接口，例如 `long key`、`int key`、`TimeWindow namespace`、`typed inner map` 等专用路径。

- RowData key 通过 **RowDataKeyAccessor** 识别为单字段 primitive、固定长度多字段或 generic bytes 三类编码方式。
- 对部分 BinaryRowData 或 bytes value, native 层支持零拷贝读取, 减少中间 byte[] 和对象重建成本。

因此, 系统的性能优化重点并不是改变 Flink 状态语义, 而是在保持语义不变的前提下, 尽量让热点访问路径走最短 JNI 路由、最窄类型表示和最少数据复制。

1.3 原生状态表组织

当前实现中, 负责权威状态存储的核心结构是 native 侧的 **StateTable<K,V>** 与其内部的 **SwissTable<K,V>**。Java 层通过状态名、serializer 和类型布局将状态注册到 native 层, 后续所有读写、清理、遍历和恢复都通过 state handle 回到对应的 **StateTable**。

状态表组织遵循以下规则:

- 每个 Flink 状态描述符在 **StateEngine** 中对应唯一的 state handle。
- 每个 state handle 对应一张 typed **StateTable<K,V>**。
- 每张 **StateTable** 按 key-group 划分内部状态分区, 与 Flink key-group 路由模型一致。
- VoidNamespace 场景下直接访问 key-group 对应的 **SwissTable**; 一般 namespace 场景下先路由到 namespace, 再进入该 namespace 独立持有的 **SwissTable**。

这种组织方式同时满足了状态隔离、快照遍历稳定性和恢复写入粒度的要求, 是整个后端一致性与可恢复性的基础。

1.4 组件解耦与集成

ForL0 State Backend 的组件划分遵循“控制面与数据面分离”的原则。Flink 侧接口、Java 状态适配、JNI 桥接和 native 数据结构各自有明确职责边界, 从而保证底层存储结构可以独立优化, 而不破坏上层状态接口语义。

核心组件的职责关系如下:

- **ForL0StateBackend** 与 **ForL0StateBackendFactory**: 负责 Flink 侧后端创建与插件注册。
- **ForL0KeyedStateBackendBuilder**: 负责 native 引擎创建、状态恢复初始化和 backend 实例构建。

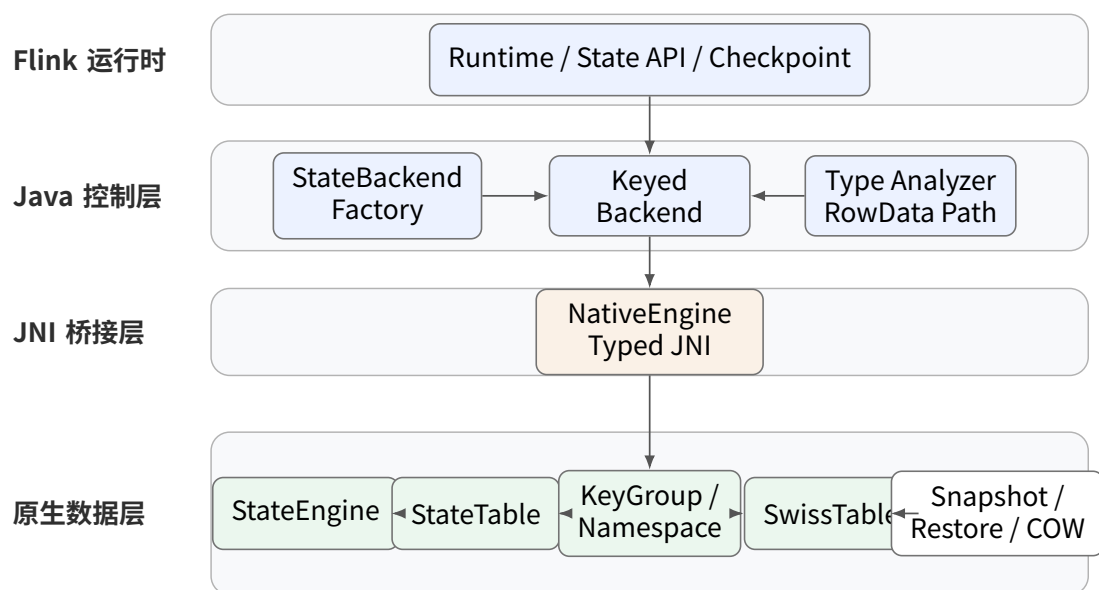


图 1: 整体架构图

- **ForL0KeyedStateBackend**: 负责管理 engine handle、state handle、状态对象缓存以及快照协调。
- **NativeEngine**: 负责定义 JNI 接口并承担共享库加载。
- **TypeAnalyzer** 与 **RowDataKeyAccessor**: 负责类型识别、布局描述生成和 RowData 快路径支持。
- **StateEngine**、**StateTable** 与 **SwissTable**: 共同构成 native 数据面，承担状态组织、访问、快照和恢复。

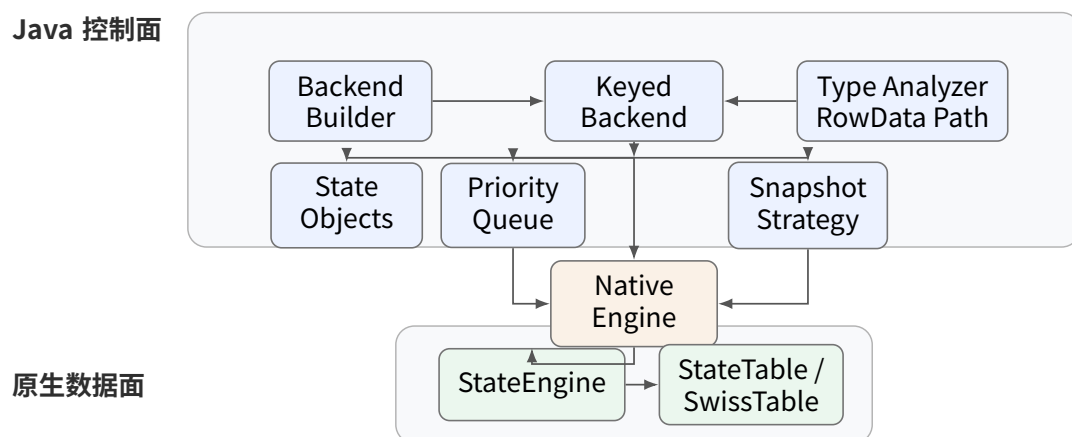


图 2: ForL0StateBackend 组件图

2. 架构设计细节

ForL0 State Backend 遵循 Flink `AbstractStateBackend` 与 `AbstractKeyedStateBackend` 的扩展方式。Java 层围绕状态描述符、serializer 和运行时上下文选择访问路径；native 层围绕 `StateTable<K,V>` 与 `SwissTable<K,V>` 组织读写、快照导出和恢复重建。

2.1 可复用组件与扩展组件

当前实现并不是完全绕开 Flink 原有状态框架，而是在与状态后端集成、优先队列管理、快照元信息和状态接口层面复用 Flink 的成熟机制，并将 Keyed State 的主数据路径替换为 ForL0 自身的 native 实现。

2.1.1 ForL0StateBackend 与 ForL0KeyedStateBackend

`ForL0StateBackend` 是 Flink 侧的后端入口，负责创建具体的 `ForL0KeyedStateBackendBuilder`。`ForL0KeyedStateBackend` 则是 Java 壳层中的核心控制组件，承担以下职责：

- 持有 native engine handle 和各状态的 native state handle。
- 缓存已创建状态对象与 state meta info。
- 在状态首次使用时完成 serializer 兼容性校验与 native 状态注册。
- 创建五类 Flink 状态实现，并把状态操作映射为相应 JNI 调用。

它在结构上是 Java 控制层与 native 数据层的直接衔接点，在语义上是 Flink 原生 `KeyedStateBackend` 的兼容实现。

2.1.2 PriorityQueueManager / HeapPriorityQueue

定时器与优先队列部分继续沿用 Flink 原生实现。`ForL0 State Backend` 不改变优先队列内部行为与定时器语义，只替换主状态的键值存储路径。因此，Priority Queue 状态仍按照 Flink 既有方式参与快照和恢复。

2.1.3 NativeEngine

`NativeEngine` 是 Java 层访问原生状态引擎的统一入口。该组件一方面负责共享库装载，另一方面定义 Java 到 C++ 的函数映射。其核心职责包括：

- 提供 `createEngine`、`destroyEngine` 等生命周期接口。

- 提供状态注册接口，用于在 native 层创建 typed **StateTable**。
- 提供 **ValueState**、**ListState**、**MapState**、**ReducingState**、**AggregatingState** 的 typed 访问接口。
- 提供快照准备、数据导出、数据恢复和快照释放接口。

当前实现中，JNI 层不保存状态本体，但它定义了 Java 控制层访问 native 数据面的全部入口。

2.2 关键设计组件

2.2.1 StateEngine

StateEngine 是 native 数据面的最顶层组件，负责管理所有已注册状态的句柄和状态表实例。JNI 接收到一次状态访问时，先根据 state handle 找到对应 **StateTable**，再结合 key-group 和 namespace 路由到最终的 **SwissTable**。

在快照阶段，**StateEngine** 统一触发所有状态表进入或退出 Copy-On-Write 视图，从而保证同一次 checkpoint 周期内整套状态共享一致的快照边界。

2.2.2 状态布局与命名空间组织

状态条目在 native 侧围绕状态表、key-group、namespace 和 **SwissTable** 展开。其设计要点如下：

- 对于 **VoidNamespace**，每个 key-group 直接对应一张 **SwissTable<K,V>**，访问路径最短。
- 对于一般 namespace，每个 key-group 下维护 namespace 到 **SwissTable<K,V>** 的映射，不同 namespace 的条目不混放在同一张表中。
- 当某个 namespace 在删除后已经为空时，系统及时移除对应空表，避免长期保留无效容器。

这种组织方式使得 checkpoint 导出、restore 重建和 namespace merge 都可以围绕统一的数据结构完成。

2.2.3 SwissTable

SwissTable<K,V> 是状态系统的底层哈希存储结构，负责维护键到值的高效映射。其设计要点包括：

- 采用 ctrl byte 加连续 slot 布局。

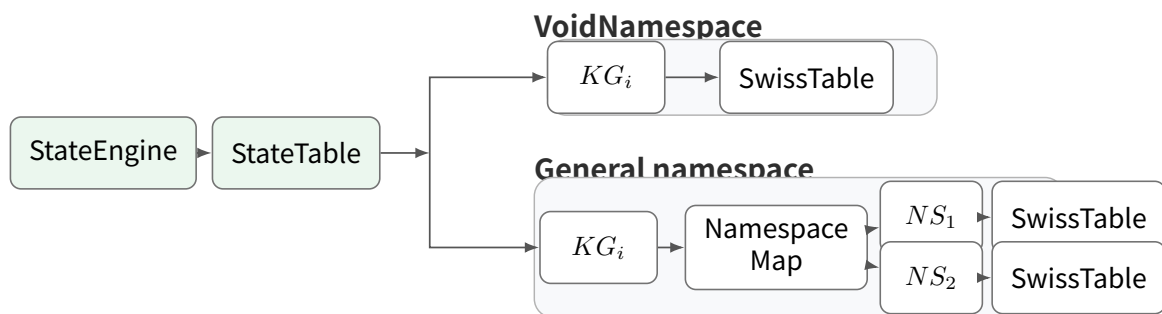


图 3: 原生状态组织结构图

- 采用 H_1 / H_2 哈希位分配方式，其中 H_2 写入 ctrl 字节， H_1 用于探测起点。
- 按 group 扫描 ctrl 区，先过滤候选槽位，再执行键比较。
- 使用 2 的幂容量、tombstone 删除和 grow/rehash 机制整理表布局。
- 命中后直接在 slot 中读取 value，不再经过额外的值地址层。

这意味着 **SwissTable** 不仅是索引结构，也是实际 value 的承载结构，从而减少二次寻址和额外缓存失配。

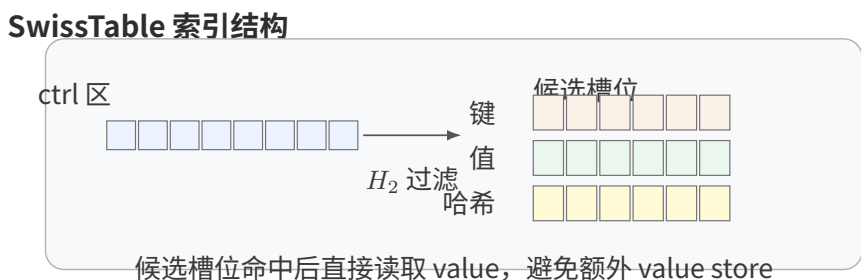


图 4: SwissTable 索引结构图

控制区与分组扫描

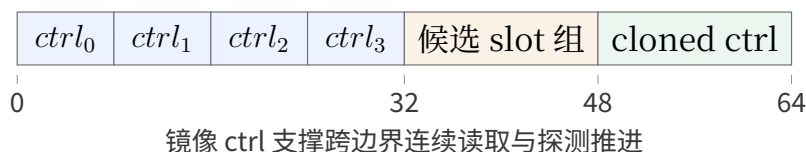


图 5: SwissTable 控制区与分组扫描示意图

2.2.4 状态访问策略与 JNI 快路径

系统通过状态访问策略和 JNI 快路径，将高频场景映射到更短、更窄、更少复制的访问路径。设计上主要包括以下几类策略：

- ValueState 根据 key 与 namespace 形态预计算 **KeyNsStrategy**。

- ListState 根据 key、namespace 和 element 类型预计算 ListStrategy。
- MapState 根据 outer key、namespace 和 inner map 形态预计算 MapStrategy。

策略一旦确定，后续每次状态访问都只进入对应分支，不再重复做整套类型判定。

2.2.5 类型分析与布局描述

TypeAnalyzer 与 native type_layout 共同构成 Java serializer 世界与 C++ 模板实例世界之间的映射层。当前重点支持的类型可分为两组：

- 基础类型：INT32、INT64、FLOAT32、FLOAT64、BOOL。
- 结构类型：STRING、BYTES、LIST、MAP、FIXED_ROW、VOID_NS 和 TIME_WINDOW。

其中，FIXED_ROW 用于固定长度的多字段 RowData key，TIME_WINDOW 用于窗口 namespace，BYTES 与 STRING 在 native 层都以连续字节容器承载，但在 checkpoint 输出时仍遵循各自 serializer 语义。

2.3 原生状态组织实现

2.3.1 结构组成

原生状态组织由四层结构组成：

- StateEngine：任务级状态引擎，维护所有状态表和状态句柄。
- StateTable<K,V>：单个状态描述符的分区状态表。
- namespace 容器：在非 VoidNamespace 模式下，将一个 key-group 下的 namespace 映射到各自的 SwissTable<K,V>。
- SwissTable<K,V>：实际保存键和值的底层哈希表。

在值组织上，slot 中直接持有 primitive、连续字节容器、列表容器、typed inner map 或 accumulator，因此结构上不再需要独立的外部 value store。

2.3.2 RowData 快路径实现

当前实现针对 RowData 提供了明确的三类路径：

- 单字段 primitive RowData key：直接解包为 long 或 int。

- 固定长度多字段 RowData key: 编码为 **FixedRow**。
- 复杂变长 RowData key: 回退到 generic 序列化路径。

对部分 RowData 或 BinaryRowData value, native 侧可以返回底层地址和长度, Java 层再构造只读内存视图, 从而减少中间 `byte[]` 拷贝。

2.3.3 主要操作算法

围绕原生索引结构, 系统的核心操作可以归纳为查询、插入、更新、删除、扩容与重哈希, 以及快照一致性视图遍历。为了与前文结构说明相呼应, 这里按照主要操作流程分别给出简要描述和伪代码, 用于说明索引结构在运行时的关键执行逻辑。

查询 (Get) 查询流程总是先定位目标状态表, 再根据 key-group 和 namespace 找到对应的 **SwissTable**。在表内查找时, 系统先扫描 ctrl 区中的 H_2 指纹以筛选候选槽位, 再对少量候选槽位执行键比较; 若当前探测组内已经出现 EMPTY 槽位, 则说明本次查询未命中, 可以直接返回空结果。

算法 1 状态查询算法

输入: 状态句柄 `stateId`, key-group `kg`, namespace `ns`, 用户键 `key`

输出: 若命中则返回对应值, 否则返回 **NOT_FOUND**

```

1: table  $\leftarrow$  StateEngine.lookup(stateId)
2: if table 为 VoidNamespace 模式 then
3:   swiss  $\leftarrow$  table.directTable(kg)
4: else
5:   swiss  $\leftarrow$  table.namespaceTable(kg, ns)
6:   if swiss =  $\emptyset$  then
7:     return NOT_FOUND
8:   end if
9: end if
10: hash  $\leftarrow$  smear(key)
11: h1  $\leftarrow$  hash  $\gg$  7, h2  $\leftarrow$  hash & 0x7F
12: group  $\leftarrow$  probeStart(h1, swiss.capacity)
13: while true do
14:   matches  $\leftarrow$  matchH2(swiss.ctrl[group], h2)
15:   for all slot  $\in$  matches do
16:     if swiss.key(slot) = key then
17:       return swiss.value(slot)
18:     end if
19:   end for
20:   if containsEmpty(swiss.ctrl[group]) then
21:     return NOT_FOUND
22:   end if
23:   group  $\leftarrow$  nextProbe(group)
24: end while

```

插入 (Put) 插入流程首先定位目标 `StateTable` 和具体 namespace 对应的 `SwissTable`。若目标 namespace 尚未建立，则系统先创建对应表结构。随后根据哈希结果执行槽位选择；若表内存在可复用槽位，则直接写入新值；若表因 tombstone 过多或增长预算耗尽而无法继续插入，则先触发 rehash 或 grow，再重试插入。

算法 2 状态插入算法

输入： 状态句柄 `stateId`, key-group `kg`, namespace `ns`, 用户键 `key`, 用户值 `value`

输出： 将 `key` 对应条目写入目标状态表

```

1: table  $\leftarrow$  StateEngine.lookup(stateId)
2: swiss  $\leftarrow$  table.getOrCreateTable(kg, ns)
3: hash  $\leftarrow$  smear(key)
4: while true do
5:   result  $\leftarrow$  swiss.put(hash, key)
6:   if result = NEED_REHASH then
7:     swiss.rehash()
8:   else if result = NEED_GROW then
9:     swiss.grow()
10:  else
11:    slot  $\leftarrow$  result & SLOT_MASK
12:    swiss.value(slot)  $\leftarrow$  value
13:    break
14:  end if
15: end while
16: if table.snapshotActive(kg) then
17:   table.recordCowMutation(kg, ns, key)
18: end if
```

更新 (Update) 更新流程与插入流程共用同一条索引定位路径。区别在于，更新场景要求条目已经存在：系统在定位到已有 slot 后，直接覆盖旧值；若当前 key 尚不存在，则更新流程退化为插入流程。对于快照窗口内的更新，系统还需要先记录旧值，再执行写回，从而保证异步快照可恢复出快照发起时刻的一致性状态。

删除 (Remove) 删除流程同样以目标 `SwissTable` 为基础执行。若目标 key 存在，则将对对应槽位标记为 tombstone；在一般 namespace 模式下，如果某个 namespace 对应表在删除后已经为空，则进一步移除该 namespace 表本身，从而避免长期持有空容器。

扩容与重哈希 (Grow/Rehash) 扩容与重哈希属于索引结构的内部整理过程。当前表如果只是 tombstone 累积过多，则可以在保持容量不变的情况下重新布局；如果有效条目数量已经逼近容量上限，则通过容量翻倍完成 grow。无论哪种情况，核心过程都是重新分配新表，并把旧表中的有效条目按新的布局重新插入。

算法 3 状态更新算法

输入： 状态句柄 *stateId*, key-group *kg*, namespace *ns*, 用户键 *key*, 新值 *value*

输出： 若条目存在则原位更新，否则按插入路径写入

```
1: table  $\leftarrow$  StateEngine.lookup(stateId)
2: swiss  $\leftarrow$  table.getOrCreateTable(kg, ns)
3: slot  $\leftarrow$  swiss.findSlot(key)
4: if slot = NOT_FOUND then
5:   调用算法 2 执行插入
6:   return
7: end if
8: if table.snapshotActive(kg) then
9:   table.recordOldValueIfNeeded(kg, ns, key)
10: end if
11: swiss.value(slot)  $\leftarrow$  value
```

算法 4 状态删除与空命名空间清理算法

输入： 状态句柄 *stateId*, key-group *kg*, namespace *ns*, 用户键 *key*

输出： 删除目标条目，并在需要时清理空 namespace 表

```
1: table  $\leftarrow$  StateEngine.lookup(stateId)
2: swiss  $\leftarrow$  table.findTable(kg, ns)
3: if swiss =  $\emptyset$  then
4:   return
5: end if
6: if table.snapshotActive(kg) then
7:   table.cowBeforeErase(kg, ns, key)
8: end if
9: swiss.erase(key)
10: if table 非 VoidNamespace 模式 and swiss.isEmpty() then
11:   table.removeNamespace(kg, ns)
12: end if
```

算法 5 SwissTable 扩容与重哈希算法

输入： 目标哈希表 *swiss*

输出： 将有效条目迁移到新的布局中，并清理 tombstone

```
1: if swiss.needGrow() then
2:   newCapacity  $\leftarrow$   $2 \times$  swiss.capacity
3: else
4:   newCapacity  $\leftarrow$  swiss.capacity
5: end if
6: fresh  $\leftarrow$  allocateSwissTable(newCapacity)
7: for slot  $\leftarrow$  0 to swiss.capacity - 1 do
8:   if swiss.ctrl(slot) 表示 FULL then
9:     hash  $\leftarrow$  swiss.hash(slot)
10:    key  $\leftarrow$  swiss.key(slot)
11:    value  $\leftarrow$  swiss.value(slot)
12:    fresh.insert(hash, key, value)
13:   end if
14: end for
15: swiss.swap(fresh)
```

快照一致性视图遍历 快照遍历阶段并不直接输出当前表内容，而是输出“checkpoint 发起时刻”的一致性视图。对于快照开始后新增的条目，遍历时直接跳过；对于快照窗口内被覆盖或删除的条目，则通过旧值记录和删除记录补写回快照输出流。这样可以在不停顿前台写入的前提下，仍然得到稳定一致的快照结果。

算法 6 快照一致性视图遍历算法

输入： 状态表 table, key-group kg

输出： 输出 checkpoint 发起时刻的一致性条目序列

```

1: for all ns ∈ table.namespaces(kg) do
2:   swiss ← table.currentTable(kg, ns)
3:   for all entry ∈ swiss.currentEntries() do
4:     if table.isSnapshotNew(kg, ns, entry.key) then
5:       continue
6:     else if table.hasOldValue(kg, ns, entry.key) then
7:       emit(ns, entry.key, table.oldValue(kg, ns, entry.key))
8:     else
9:       emit(ns, entry.key, entry.value)
10:    end if
11:  end for
12:  for all deleted ∈ table.deletedEntries(kg, ns) do
13:    emit(ns, deleted.key, deleted.value)
14:  end for
15: end for

```

2.4 ForL0State 实现

2.4.1 ForL0ValueState

ForL0ValueState 负责单值状态的读取、写回与清理。其设计重点在于根据 key、namespace 和 value 的组合，尽量选择最短路径完成一次状态访问。对于 primitive 值，会优先使用一次 JNI 调用完成存在性判断与取值；对于 bytes 和 string，会优先使用连续字节返回；对于 RowData，则优先判断能否走零拷贝或压缩编码路径。

2.4.2 ForL0ListState

ForL0ListState 负责列表状态的读取、追加和整体更新。对于 long key 加 long element 组合，系统使用专用路径减少逐元素反序列化；对于 generic element 类型，仍使用 serializer 路径保证兼容性。namespace merge 在语义上遵循 Flink 规则，只是底层值容器保持在 native 侧。

2.4.3 ForL0MapState

ForL0MapState 的关键设计在于 typed inner map。对常见组合，系统允许按 inner entry 粒度执行 get、put、remove 和 contains，而不是把整张 map 始终当

作 opaque bytes blob 读写。只有在 generic 回退路径下，才需要通过通用序列化方式处理内层 map。

2.4.4 ForL0ReducingState 与 ForL0AggregatingState

这两类状态采用“native 存储，Java 执行用户函数”的模式。对于 `ReducingState`，旧值保存在 native 状态中，Java 侧执行 `ReduceFunction.reduce`；对于 `AggregatingState`，accumulator 保存在 native 状态中，Java 侧执行 `AggregateFunction` 的 `createAccumulator`、`add`、`merge` 和 `getResult`。这种分层保证了用户函数语义不变，同时让状态本体继续停留在 native 数据面。

3. 容错机制设计

ForL0 State Backend 的容错机制遵循 Flink Checkpoint / Savepoint 的一致性语义。系统的 KV 状态快照来源于 native `StateTable` 的一致性视图，Priority Queue 状态仍沿用 Flink 原生写出机制，最终共同组成完整的 Keyed State 快照结果。

3.1 快照发起

在 Flink 发起 snapshot 后，系统首先进入同步准备阶段。Java 层通过 `ForL0SnapshotStrategy` 调用 `NativeEngine.prepareSnapshot`，native `StateEngine` 随后将所有相关 `StateTable` 切换到 Copy-On-Write 模式。与此同时，Java 层构造 `ForL0SnapshotResources`，收集 KV state meta info、priority queue snapshot 资源以及异步写出所需的上下文。

这一阶段的目的是不是执行序列化，而是建立一份稳定的“快照边界”，保证异步写出线程读取的是 checkpoint 发起时刻的一致性视图。

3.2 快照采集

快照采集阶段按 key-group 写出数据，主要流程如下：

1. 写出 `KeyedBackendSerializationProxy`，记录 key serializer snapshot 与 state meta info snapshot。
2. 为每个 key-group 记录输出偏移并写出 priority queue 状态块。
3. 调用 native checkpoint 写出逻辑，导出该 key-group 下全部 KV state 数据。
4. 所有 key-group 完成后，调用 `NativeEngine.releaseSnapshot` 释放快照上下文。

native KV block 的内部组织以 state 为单位记录 stateId、entryCount 和具体条目内容。namespace、key 和 value 的输出格式与 Flink serializer 语义保持兼容，从而保证 restore 端可以按统一格式重建状态。

3.3 Copy-On-Write 一致性模型

当前实现的 Copy-On-Write 不是复制整张状态表，而是记录“自快照开始之后发生变化的条目”。其基本思想如下：

- 快照开始时，在每个 key-group 上打开快照活动标记。
- 若某个条目在快照窗口内第一次被修改，则先保存旧值，再执行更新。
- 若某个条目是在快照开始后新增，则记录为“快照后新增”，本轮快照中不输出。
- 若某个条目在快照窗口内被删除，则记录其快照时刻旧值，以便异步遍历时仍能写出。

异步遍历阶段并不是简单遍历当前表，而是在当前表内容之上叠加覆盖记录、删除记录和新增长记，从而还原出“快照发起时刻”的一致性视图。对于一般 namespace 状态，上述过程按 namespace 分别进行。

3.4 Savepoint 与状态恢复

ForL0 State Backend 同时支持普通 checkpoint 恢复与 canonical savepoint 恢复。恢复流程由 ForL0KeyedStateBackendBuilder 统一驱动，主要包括以下步骤：

1. 读取状态句柄并恢复 KeyedBackendSerializationProxy。
2. 执行 key serializer 兼容性校验，重建 state meta info registry。
3. 恢复 priority queue 状态。
4. 按 key-group 读取 KV 数据块，并写回对应 typed StateTable。
5. 对 canonical savepoint，将规范格式条目翻译为 native 状态表中的键值结构。

恢复完成后，backend 重新持有全部 engine handle 与 state handle，后续访问继续通过 Java 控制层和 JNI 路由到 native 数据面。

3.5 资源回收与异常处理

为避免 native 资源泄漏，当前实现通过显式生命周期控制保障异常路径下的资源回收：

- snapshot 正常完成后释放 Copy-On-Write 快照状态。
- snapshot 失败时，异常路径仍尝试释放快照上下文。
- backend 构建或 restore 失败时，builder 负责销毁已创建的 native engine。
- backend dispose 时调用 `NativeEngine.destroyEngine` 释放状态表、容器和内部缓存资源。

4. 插件式集成设计

ForL0 State Backend 以插件方式接入 Flink。系统通过 `StateBackendFactory` 完成后端工厂注册，通过 JNI 动态加载 native 状态引擎，并通过 Flink 标准 backend 构建流程完成运行时接入。

4.1 加载机制

ForL0 的加载机制分为两部分：

1. Flink 通过 `ForL0StateBackendFactory` 创建 `ForL0StateBackend` 实例。
2. `ForL0KeyedStateBackendBuilder` 在构建阶段调用 `NativeEngine.ensureLoaded()` 装载 native 共享库。

native 共享库的加载采用两级策略：优先从系统库路径加载；若系统库路径不可用，则从打包资源中提取后再加载。这一设计使得开发、测试和交付部署可以共用同一套接口，而不需要在上层状态代码中引入额外分支。

4.2 运行时接入边界

在运行时接入方面，ForL0 State Backend 保持了以下边界约束：

- 对 Flink 上层用户代码透明，用户状态访问方式不因后端替换而改变。
- 对 Flink 内核源码无侵入，不修改 Flink 原生状态接口语义。
- Priority Queue 保持原有行为，Keyed State 则切换到 ForL0 自身的 native 数据面。

因此，系统的集成重点在于状态后端实现与运行时生命周期协调，而不是对用户 API 做扩展。

5. 总结

ForL0 State Backend 当前已经形成一套清晰的三层架构：Java 控制层负责接口适配、状态注册和用户函数执行；JNI 桥接层负责共享库加载和类型专用访问入口；C++ 原生状态引擎负责权威状态存储、命名空间组织、SwissTable 哈希结构以及快照恢复。

从设计角度看，当前版本相对早期方案的最大变化，在于状态主数据面已经完全收敛到 native 侧。围绕这一调整，系统进一步形成了 typed StateTable、RowData 快路径、typed inner map、TimeWindow 专用路径以及 native Copy-On-Write 快照机制。这些设计共同支撑了在保持 Flink 语义兼容的前提下，对热点状态访问路径进行更彻底、更工程化的优化。

对于项目交付而言，本文档的核心结论是：ForL0 State Backend 并不是对现有 Flink 状态存储的局部修补，而是围绕统一 native 数据面完成的一次结构性重构。该架构既保持了与 Flink 状态接口和容错机制的兼容，也为后续在高性能哈希组织、原生内存利用和硬件友好访问路径上的持续优化保留了清晰扩展空间。